CAST — 이야기 정리

전체판(CAST_JOK_v2_전체판) 36 쪽의 이야기 흐름을 몇 쪽으로 줄인 것. 숫자는 전체판과 같다. 2026-10-19.

문제와 아이디어

한 줄

CAST 는 리전 용량을 넘지 않고 마감을 지키면서 탄소 배출을 줄이는 스케줄러다. 매시간 기다리는 작업 전부의 리전과 시작 시각을 다시 정하고, 지금 시작할 작업만 확정한다.

왜 필요한가

같은 1 kWh 라도 어디서, 언제 쓰느냐에 따라 나오는 탄소가 다르다. 본 연구의 여덟 리전에서 2025 년 탄소집약도의 연평균은 인도와 프랑스가 약 40 배 차이 난다. 그래서 작업을 깨끗한 리전과 시각으로 옮기면 같은 계산을 더 적은 배출로 마칠 수 있다.

그런데 용량을 생각하지 않으면 모든 작업이 가장 깨끗한 한 곳으로 간다. 선행 연구(Sukprasert 등)의 배치를 우리 1 년치 자료에 그대로 돌리면 작업의 99.75%가 프랑스 한 리전으로 가고, 그 리전은 동시에 129 건을 돌려야 한다. 평소(홈 리전에서 바로 실행) 어느 리전도 13 건을 넘지 않으니 약 10 배다. 가장 깨끗한 리전의 가장 깨끗한 시각은 모든 작업에게 같기 때문에 생기는 일이다.

아이디어

- 용량을 모든 대기 작업이 함께 나눠 쓰는 제약으로 두고, 리전과 시각을 하나의 선형 계획으로 함께 정한다.
- 미래 자리를 미리 확정하지 않는다. 매시간 다시 풀고, 지금 시작하는 작업만 확정한다(운영연구의 롤링 지평, 제어의 모형 예측 제어와 같은 구조).
- 그래서 "용량을 무엇과 함께 확인하느냐"가 위반을 가른다. 다른 작업의 확정 계획을 보지 않고 시각을 정하면 넘치고, 계획을 모두 보거나, 확정하지 않고 다시 풀거나, 시작 때 확인해 빈 리전으로 보내면 넘치지 않는다.

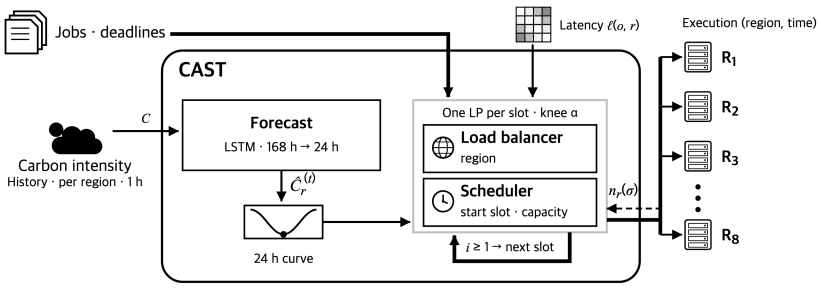


그림 1. CAST 파이프라인: 예측 → 매시간 대기 작업 전체의 리전·시각을 다시 풀기 → 지금 시작하는 작업만 확정

증거

핵심 숫자 (전체판과 같은 값)

질문	답	전체판 위치
CAST 는 얼마나 줄이나	평균 배출계수로 합성 71.93%(이용률 8.4%), Azure 51.35%(26.1%, 예약 없이, 전송 제외). 대부분 리전 이동에서 온다(합성 공간만 61.57%). 모든 작업 마감 안, 상한을 넘긴 배정 0	4.3·4.7
용량을 지키는 비용은	합성에서 상한 없는 실행 절감의 98%(73.39% 대 71.93%)	4.3
확인 방식이 위반을 가르나	다른 작업의 확정 계획을 보지 않고 시작 때 확인하지 않으면 합성 9.0~17.0%(Azure 11.4~14.7%) 위반, 계획을 보거나·다시 풀거나·빈 리전으로 보내면 0	4.4
가장 가까운 설계보다 나은가	같은 지연에서 다시 풀지 않는 설계(Zanotto 식)보다 합성 0.96%p, 다시 뽑은 네 벌 1.23~1.56%p(고정 α 0.575), Azure 2.53%p 덜 배출. 완전예지 1.01%p, 미래 정보를 모두 뺀 설정 0.96%p. 리전별로 나누면 프랑스에 더 몬 것이 아니라 리전마다 더 깨끗한 시각을 다시 고른 데서 온다	4.4·표 8
한 점만의 결과인가	곡선 전체에서 지연 30 ms 위면 0.9~2.1%p 위, 10 ms 아래는 겹침	4.4·그림 7
상한에 따라 다른가	다시 푸는 몫은 깨끗한 리전의 상한이 빠듯할수록 크다. 상한 13 에서 1.46%p, 리전별 상한 2.10%p, 넉넉한 상한 26 에서는 차이 없음(0.34%p, 신뢰구간 0 포함). 시작한 작업만 확인형 위반은 4.8~15.0%, CAST 는 0	4.4
선행 정책과 비교하면	더 많이 줄이는 선행 정책 셋(둘은 1 년치 탄소를 아는 상한 분석)은 가장 붐빈 리전에 평소의 2.2~9.9 배를 요구. CASPER 에 같은 상한 17 을 걸면 78.53%로 CAST(탄소만) 95.65%보다 낮지만, 같은 상한의 공간만도 93.96%라 이 차는 용량 인지의 몫이 아니다	4.8·표 14
부하가 높으면	Azure 상한을 조여 이용률 53.5%까지 위반 0, 66.1%에서 0.32%(모두 그 순간 모든 리전이 차 옮길 자리가 없을 때)	4.4
회계를 바꾸면	전송·전과정·PUE·대칭 근사에서는 우위가 남는다. 프랑스만 바꾼 한계 근사에서는 합성 두 짝이 뒤집히고(−0.89, −2.15%p), 불리한 가정을 다 겹치면 합성에서 CAST 가 1.49~3.13%p 더 배출(Azure 는 도착 고정형보다 0.89%p 앞서고 대피형과 차이 없음). 프랑스를 가스 발전 수준(약 400 g)으로 두는 대칭 근사에서는 무릎점이 21.20%로 준다	4.1·표 12

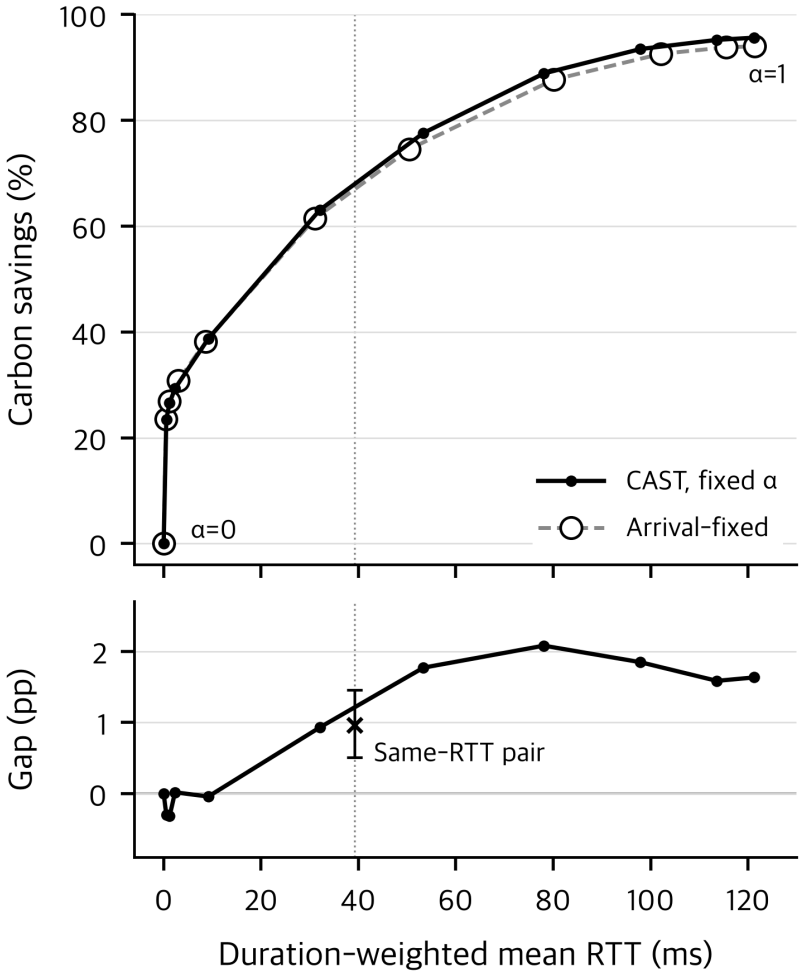


그림 7. 같은 지연에서 본 두 설계(CAST 와 다시 풀지 않는 도착 고정형)의 고정 가중치 곡선 — 지연 30 ms 위에서 CAST 가 0.9~2.1%포인트 위

한 문장으로

CAST 는 위반 없이 줄이고, 같은 지연에서 가장 가까운 설계보다 약 1%p 더 줄이며, 그 이점은 미래 정보가 아니라 새로 도착한 작업과 함께 다시 짜는 데서 오고 용량이 빠듯할수록 커진다.

대가와 한계, 면담에서 정할 것

대가

· 이점이 작다. 같은 지연에서 약 1%p(합성), Azure 2.5%p 다.

· 기본 운용점(무릎점, 슬롯마다 가중치를 고르는 규칙)은 지연 목표 없이 첫 슬롯부터 쓸 수 있지만, 같은 지연의 고정 가중치보다 1.77~2.64%p, 첫 주 자료로 고른 고정 가중치보다 약 0.9%p 더 배출한다. 무릎점으로 견주면 다시 풀지 않는 설계와 배출 차가 신뢰구간 안이고, 대기는 25% 짧다.

· Azure 에서 대기 시간은 CAST 가 더 길다(2.51 대 1.81 시간).

· CAST 는 시작하기 전까지 시작 시각을 사용자에게 알려 줄 수 없다. 다시 푸는 대가로 계산이 4~11 배 들지만 슬롯당 1 초 안이다.

한계 (전체판 5 장)

· 예약 설정에 따라 Azure 결과가 뒤집힌다(준비 구간 규칙의 예약 13 이면 다시 풀지 않는 설계가 2.10%p 앞섬). 13 은 계속 도는 VM 을 두 번 센 값이라 본문은 예약 없이 쓴다.

· 평균 배출계수로 셌다. 프랑스로 옮긴 몫만 이웃 망 기준으로 세면 순서가 뒤집혀 다시 풀지 않는 설계가 0.89%p 앞선다(대칭으로 세면 CAST 가 1.19%p 앞서지만 이 근사는 분모 하한에 크게 좌우된다).

· 실제 클러스터가 아닌 시뮬레이션이고 이론 보장이 없다. PUE 는 1.1~1.6 민감도로만 보았고 유휴 전력·추가 설비의 내재 배출은 넣지 않았다. 지연 한도는 주 실험에서 걸리지 않아 결과가 공간 자유도에 지배된다.

· 선행 정책은 원 논문의 자료가 아니어서 원 수치를 재현하지 못했다(설정 대응은 표 13).

면담에서 정할 것

· 이점이 약 1%p 인 시스템 논문으로 낼지(위반 0 과 필요 용량 비교가 주된 어필).

· 대표 운용점: 무릎점을 유지할지, 첫 주 고정 가중치(0.9%p 더 줄임)로 바꿀지.

· 투고본 분량: 전체판으로 낼지, 다시 줄일지.

· 자세한 안건은 면담 안건 문서(CAST_v2_면담안건)에 있다.